

Unit 4.2 – SkipList

CS 201 - Data Structures II

Spring 2023

Habib University

Syeda Saleha Raza

Skiplist

- Conceptually, a skiplist is a *sequence of singly-linked lists* $L_0, L_1, L_2, \dots, L_h$.
- Each list L_r contains a subset of the items in L_{r-1} .
- The items in L_r are obtained by tossing a coin for each element, x , in L_{r-1} and including x in L_r if the coin turns up as heads. This process ends when we create a list L_r that is empty.

Skiplist

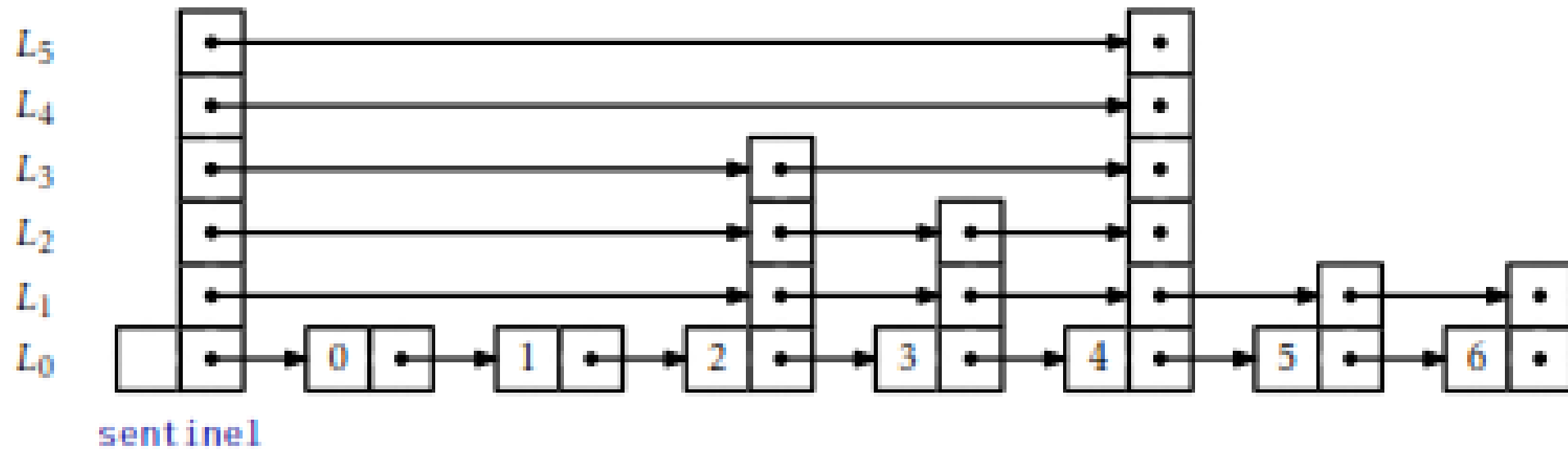


Figure 4.1: A skiplist containing seven elements.

Finding an element in Skiplist

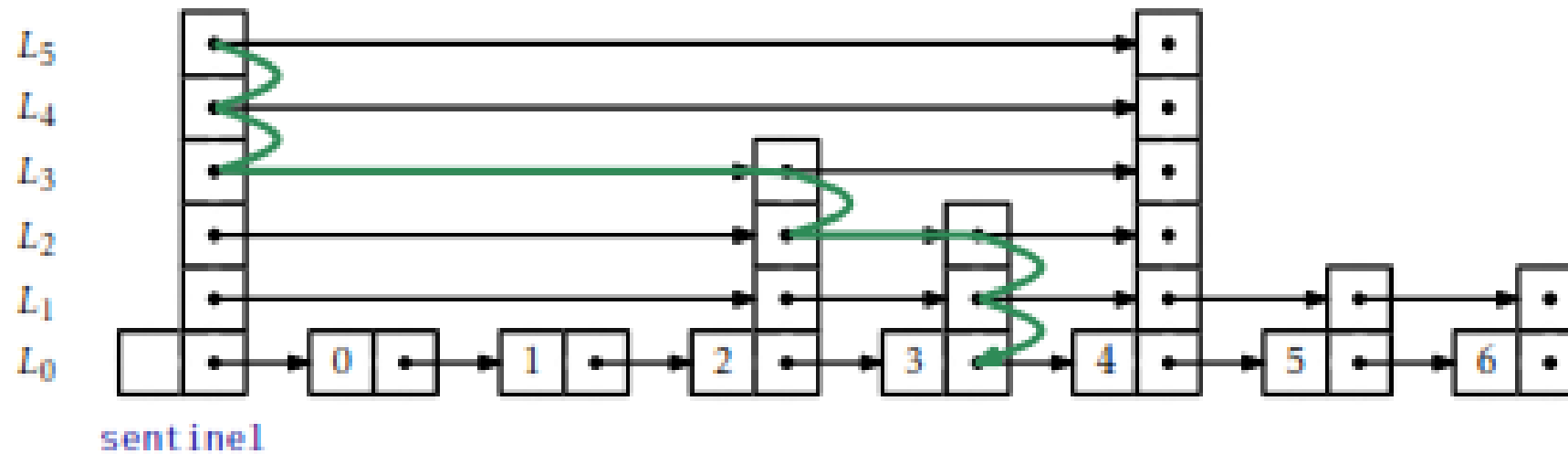


Figure 4.2: The search path for the node containing 4 in a skiplist.

Example – Building a skiplist

Numbers to be added to the list in the given order:

3 8 1 14 6 11 9 21 12

Coin Outcome

H	H	T	T	H	T	T	H	T	H	H	T
T	H	H	H	T	H	T	T				

Exercise – Building a skiplist

- Let's construct a skiplist with these values:
 - 7, 8, 9, 4, 3, 6, 1, 5
- For each element, here is the number of times the coin turn up as head before coming up with tail:
 - 3, 2, 0, 4, 2, 2, 0, 3

SkiplistSet

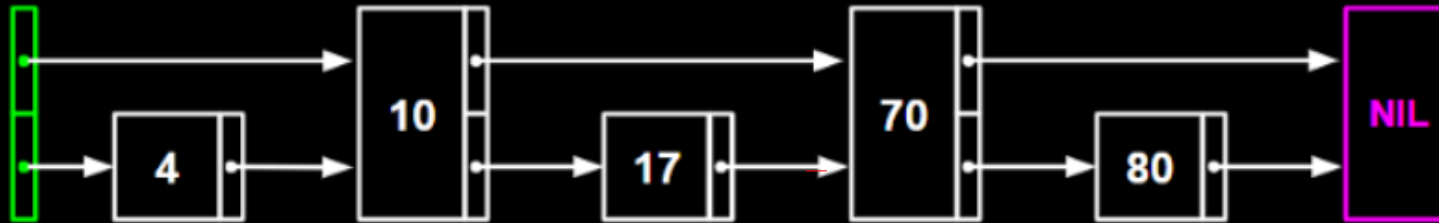
```
struct Node {  
    T x;  
    int height;    // length of next  
    Node *next[];  
};
```

Finding an element

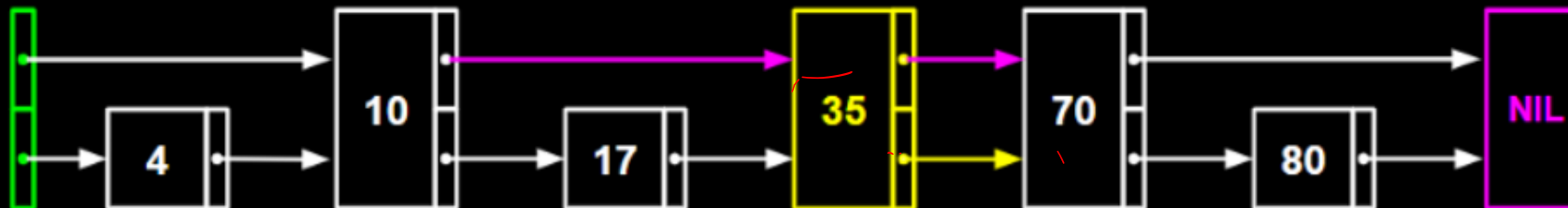
```
SkiplistSet
Node* findPredNode(T x) {
    Node *u = sentinel;
    int r = h;
    while (r >= 0) {
        while (u->next[r] != NULL
                && compare(u->next[r]->x, x) < 0)
            u = u->next[r]; // go right in list r
        r--; // go down into list r-1
    }
    return u;
}
T find(T x) {
    Node *u = findPredNode(x);
    return u->next[0] == NULL ? null : u->next[0]->x;
}
```

Skip List - Insertion

How to Insert 35 ?

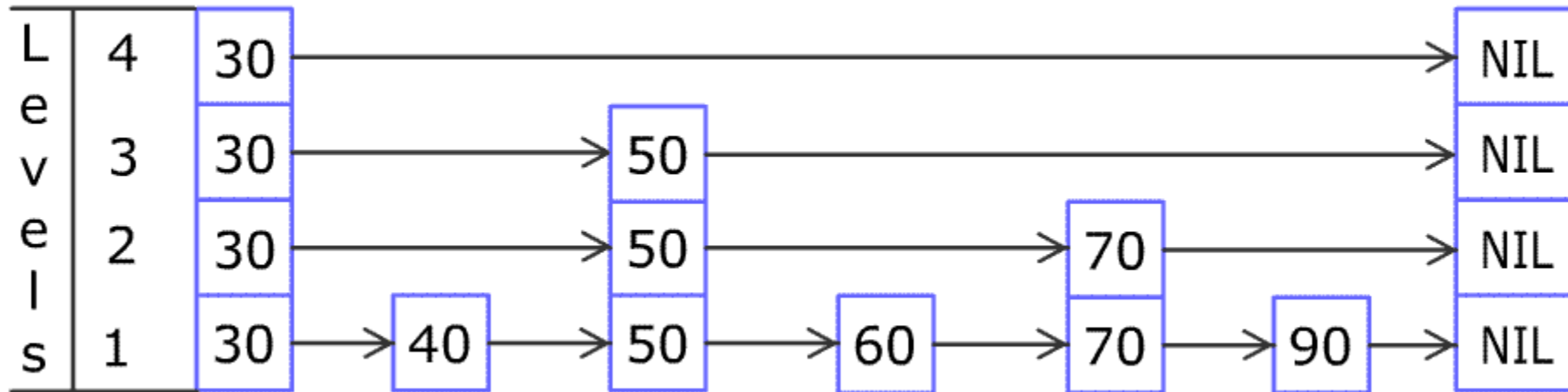


HEADS [add
1 level]



TAILS
[terminate]

Skip List - Insertion



Skip list – Insertion

- keeping track of pointers using Stack

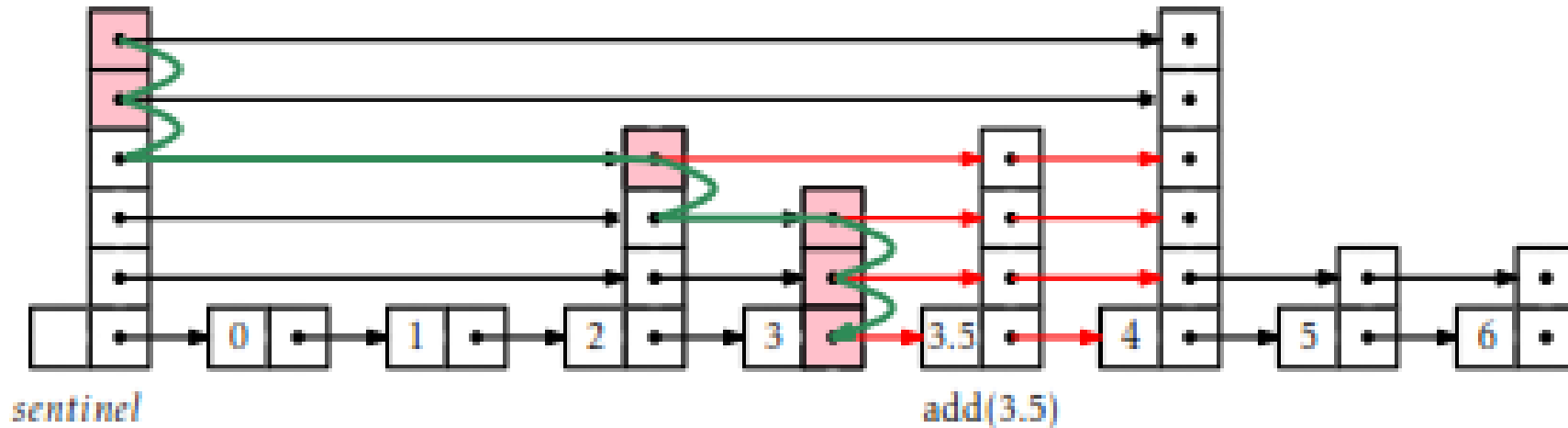


Figure 4.3: Adding the node containing 3.5 to a skiplist. The nodes stored in *stack* are highlighted.

Skip List - Deletion

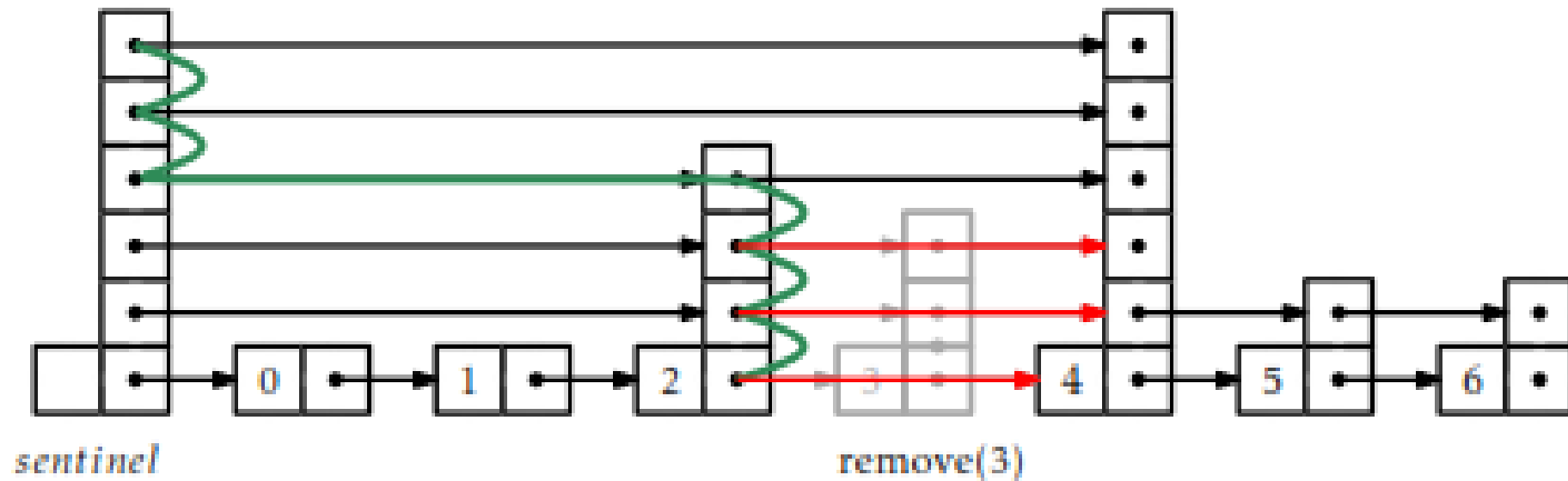
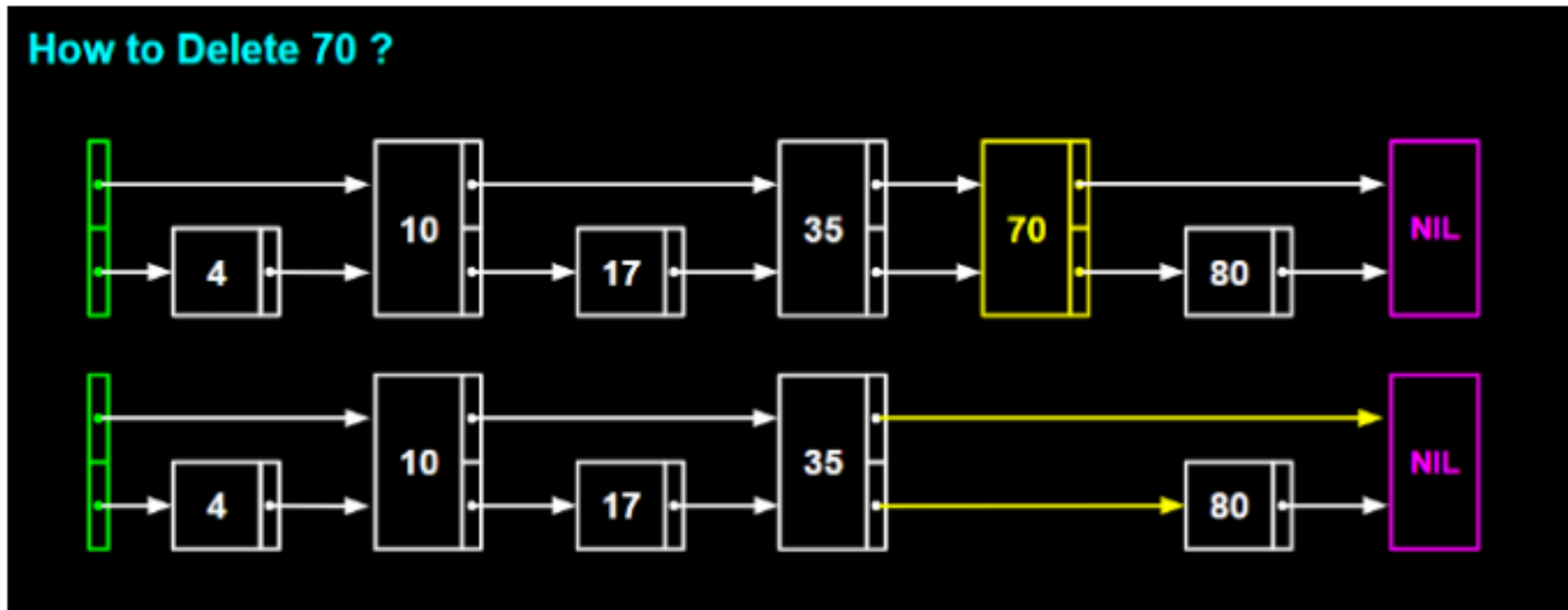


Figure 4.4: Removing the node containing 3 from a skiplist.

Skip List - Deletion



Implementing SSet using SkipList

Implementing List interface using Skiplist

SkiplistList

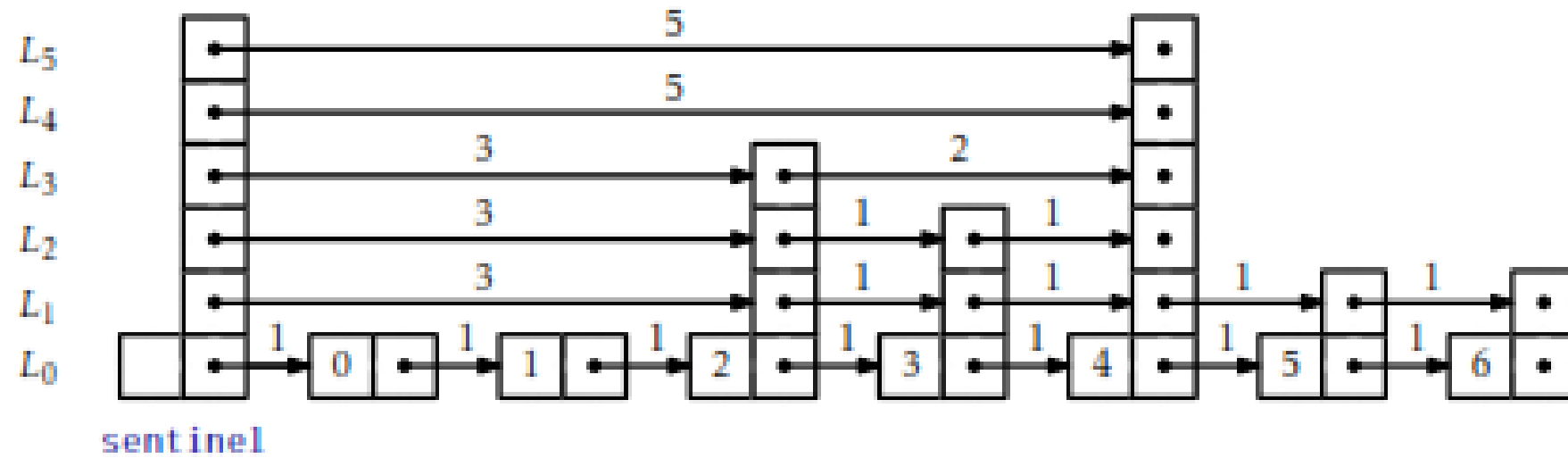


Figure 4.5: The lengths of the edges in a skiplist.



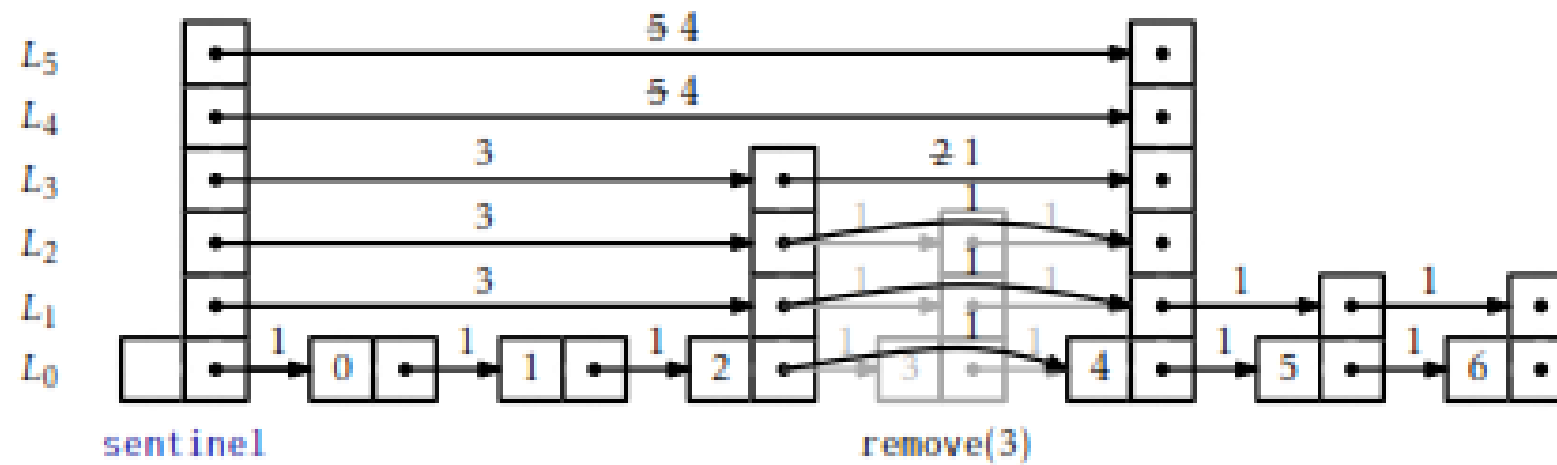
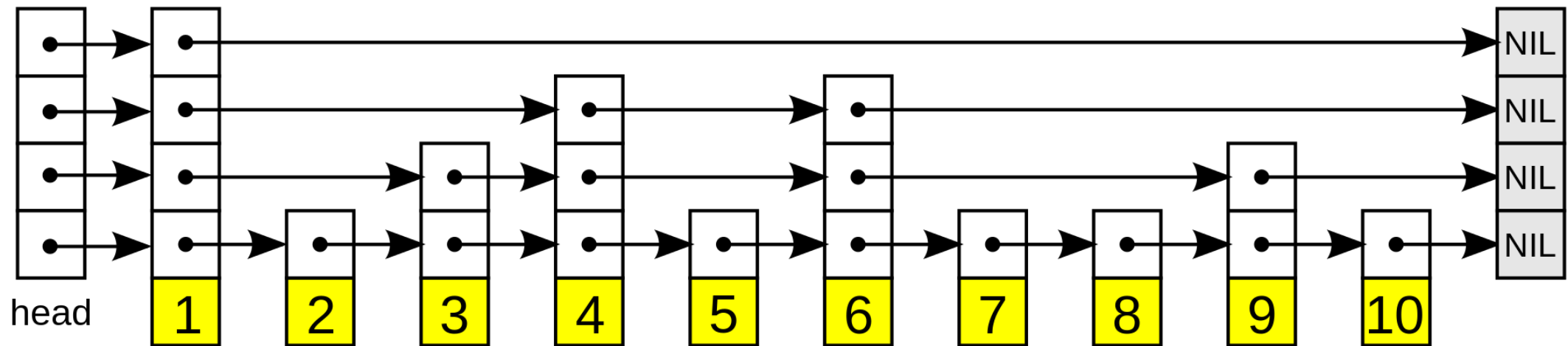


Figure 4.8: Removing an element from a `SkiplistList`.

Analysis of Skip List



How many times do you think you will toss the coin (at max) to insert n elements?

Lemma 4.2. *Let T be the number of times a fair coin is tossed up to and including the first time the coin comes up heads. Then $E[T] = 2$.*

How many nodes do you expect in the linked list including or excluding the sentinel nodes?

Lemma 4.3. *The expected number of nodes in a skiplist containing n elements, not including occurrences of the sentinel, is $2n$.*

Lemma 4.5. *The expected number of nodes in a skiplist containing n elements, including all occurrences of the sentinel, is $2n + O(\log n)$.*

Expected height of skiplist containing n nodes?

Lemma 4.4. *The expected height of a skiplist containing n elements is at most $\log n + 2$.*

Lemma 4.5. *The expected number of nodes in a skiplist containing n elements, including all occurrences of the sentinel, is $2n + O(\log n)$.*

The complexity of skiplist Search

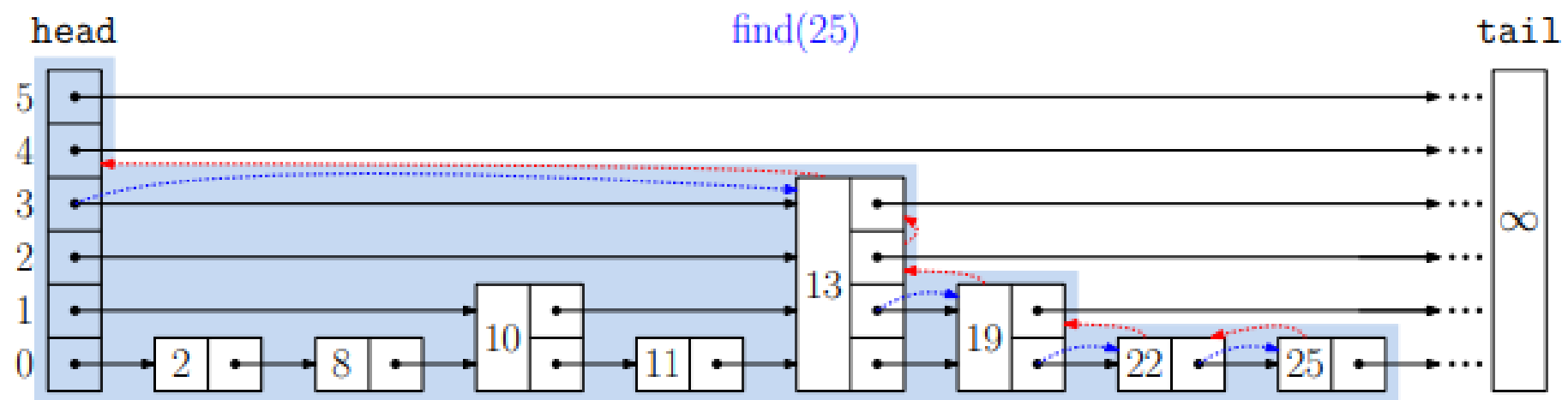


Fig. 4: The search path (blue) to $x = 25$ and the reverse search path (red).

Complexity of Skiplist Search

- let $E(i)$ denote the expected number of nodes visited in the skip list at the top i levels of the skip list.
 - In the given skiplist, $E(1)$ would be the expected number of steps taken at level 5
 - $E(2)$ would be the expected number of steps taken at top 2 levels (i.e. levels 4 and 5)
 - $E(3)$ would be the expected time spent in top 3 levels (i.e. levels 3--5)
 - $E(6)$ would be the expected number of steps at all the levels

Complexity of Skiplist Search

- Our analysis is based on walking backwards along the search path.
- Whenever we arrive at some node of level i , the probability that it contributes to the next higher level is exactly p , and if so, the search path came from above.
- On the other hand, with the remaining probability $1 - p$, this node stopped at this level, and the backwards search stays at the same level.
- Counting the current node we are visiting (+1), we can express $E(i)$ by the recurrence:

$$E(i) = 1 + p E(i-1) + (1-p) E(i)$$

By substitution, we can see that

$$E(i) = i/p$$

Where i is the number of levels i.e. $O(\log n)$

Lemma 4.6. *The expected length of a search path in a skiplist is at most $2 \log n + O(1)$.*

Readings

- ODS: Chapter 4

Resources

- Open Data Structures (pseudocode edition), by Pat Morin. Available online at <http://opendatastructures.org>
- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing
- <https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation>
- <https://www.javatpoint.com/skip-list-in-data-structure#:~:text=The%20skip%20list%20is%20used,known%20as%20a%20skip%20list.>

Thanks